

- $\hat{C}_{t,1}$: Cost estimate for all the first attempts (calculated using the bootstrapping method).
 - $\hat{C}_{t,2}$: Raw estimated cost of the additional attempts for a given task type (t).
 - $C_{t,2}$: Total cost accumulated across the additional attempts.
 - D : Sum of all raw estimated costs for all task types used as a denominator used to scale the cost estimate for the additional attempts.
 - n_t : Number of additional attempts per task type.
 - N_t : 40 (the number of tasks per task type).
 - $\text{Err}(\cdot)$: Margin of error of the enclosed quantity.
- **Calculate Margin of Error of Estimated Total Cost:** We assumed independence between the task-level cost estimates for simplicity. Using first-order error propagation, we computed the margin of error for the total cost associated with each task type and information setting using the following formulas:

$$\text{Err}_t(\hat{C}_{t,\text{total}}) = \sqrt{\text{Err}_t(\hat{C}_{t,1})^2 + \left(\frac{C_{t,2}}{D} \cdot \text{Err}_t(\hat{C}_{t,2})\right)^2 + \left(\frac{\hat{C}_{t,2} \cdot C_{t,2}}{D^2} \cdot \text{Err}_t(D)\right)^2} \quad (4)$$

$$\text{Err}_t(\hat{C}_{t,2}) = \left| \frac{n_t}{N_t} \right| \cdot \text{Err}_t(\hat{C}_{t,1}) \quad (5)$$

$$\text{Err}_t(D) = \sqrt{\sum_t \left(\text{Err}_t(\hat{C}_{t,2})\right)^2} \quad (6)$$

F The Meaning of the Economic Impact of BountyBench

One of the key design decisions in BountyBench is to select tasks with economic value to help assess the economic impact of AI agents in cybersecurity, as opposed to simply solving logic problems in a vacuum. Here, the economic value assigned to each task is the amount that was paid out or would have been paid out to human experts completing the tasks. Accordingly, it suggests that AI agents could potentially complete tasks with similar payouts in the wild, with a few considerations. First, to be awarded the bug bounty, humans must manually inspect and award the prize money; this may take into consideration factors besides correctness, including communication, and requires writing up a report as well (for disclosure bounties). Second, a bounty is awarded only once for a specific bug so agents would no longer be awarded money for these particular bugs, though one would assume that capabilities on these generalize to new bugs. Third, patches need to not only fix the vulnerability and pass invariants, but also seem reasonable under human scrutiny and review. Fourth, patches may not always be available, and typically can be claimed by either the bug bounty hunter disclosing the initial bounty or the organization given the non-public disclosure period.

More broadly, we have seen other evidence that AI agents can make an economic impact in this domain. Most notably, XBow, a startup that focuses on building AI agents for cybersecurity, announced that their agent reached the top spot on the US leaderboard of HackerOne [34]. This involved their agent completing real world bug bounty tasks, similar to the tasks measured on BountyBench. We have seen other evidence of this with Google’s Big Sleep [5] and the DARPA AIXCC challenge [9], which have been more focused on capability than economic impact.

To provide more concrete grounding, we analyze the net profit per unit time for each agent, when subtracting API and infrastructure costs. Naively, we see that the economics of patching code is considerably better than detection, with up to \$32.39/min with Claude Code. However, patching is likely an overestimate given that it may introduce new vulnerabilities or performance regressions, and may not be available unless someone detects the vulnerability to begin with. In contrast, we see that the economics of detection is significantly less favorable, with multiple agents not breaking even and OpenAI Codex CLI: o4-mini having the best value at \$12.82/min.

Table 9: Net profit per unit time for *Detect* and *Patch*

Agent	Detect (\$/min)	Patch (\$/min)
Claude Code	+3.61 $\pm$ 0.006	+32.39 $\pm$ 0.009
OpenAI Codex CLI: o3-high	+6.91 $\pm$ 0.004	+20.17 $\pm$ 0.002
OpenAI Codex CLI: o4-mini	+12.82 $\pm$ 0.004	+18.35 $\pm$ 0.001
C-Agent: o3-high	-0.35	+3.14
C-Agent: GPT-4.1	-0.10	+5.87
C-Agent: Gemini 2.5	+0.95	+2.85
C-Agent: Claude 3.7	+0.71	+10.45
C-Agent: Qwen3 235B A22B	-0.00	+1.11
C-Agent: Llama 4 Maverick	-0.01	+10.08
C-Agent: DeepSeek-R1	+0.01	+2.81

As stated in Appendix E, we do not assess and value Exploit, as it is not assigned independent economic value, and does not account for additional care needed to ensure patches satisfy reviewer requirements. We also do not assess Detect with information, which would not be provided when running the agents autonomously.

G Agent Details

To assess the cybersecurity capabilities of agentic systems, we evaluated 10 agents: 3 coding agents (Claude Code and OpenAI Codex CLI with o3-high and o4-mini) and 7 custom agents (C-Agent: o3-high, C-Agent: GPT-4.1, C-Agent: Gemini 2.5, C-Agent: Claude 3.7, C-Agent: Qwen3 235B A22B, C-Agent: Llama 4 Maverick, and C-Agent: DeepSeek-R1).

All agents were run with a single submission attempt, with up to three attempts on each task.

G.1 Coding Agents

Claude Code.

Claude Code is “an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster through natural language commands”, built by Anthropic [4]. Its key capabilities include “editing files and fixing bugs across your codebase” and “searching through git history, resolving merge conflicts, and creating commits and PRs”. We executed Claude Code with Claude 3.7 Sonnet [3] (claude-3-7-sonnet-20250219) directly via a bash command in the Kali Linux environment by running:

```
claude -d -p {prompt} --output-format json -verbose
--allowedTools "Bash,Edit,Write,Agent,Glob,Grep,LS,Read,NotebookEdit,
NotebookRead,WebFetch"
```

The agent was run with no iteration limit and no token limit. The agent ran until it determined the task was complete.

OpenAI Codex CLI.

OpenAI Codex CLI is “a lightweight coding agent that can read, modify, and run code on your local machine to help you build features faster, squash bugs, and understand unfamiliar code”, built by OpenAI [28]. It “runs entirely in the terminal, which is perfect for quick iteration without switching contexts”. We executed OpenAI Codex CLI with o3-high [29] (o3-2025-04-16 with “high reasoning effort”) and o4-mini [29] (o4-mini-2025-04-16) directly via a bash command in the Kali Linux environment by running:

```
codex --quiet --json --dangerously-auto-approve-everything
--model {model} --approval-mode full-auto {prompt}
```

The agent was run with no iteration limit and no token limit. The agent ran until it determined the task was complete.

Prompting. Both coding agents received identical prompts to those used for the custom agents, with one exception: we removed the structured response format section. This omitted the required output fields such as Reflection, Plan and Status, Thought, Log, and Command.

G.2 Custom Agents

We draw on the Cybench agent design, which follows an act, execute, update loop, where it acts based on its memory, the action is executed in the environment, it updates its memory based on the observation from the execution, and continues in a loop until finalizing its submission [38]. As discussed in Cybench, “the agent response contains 5 fields: (1) **Reflection**, intended for the agent to reflect about the last observation. (2) **Plan and Status**, intended for the agent to plan and keep track of current status at a high level. (3) **Thought**, intended for the agent to think before it acts to have more a reasoned action. (4) **Log**, intended to help the agent plan based on its past actions and observations. (5) **Command**, a bash command that will be executed as is in the environment.” A special command, `FinalSubmissionCommand`, triggers performance evaluation and termination of the current task.

We run custom agents with the following 7 models: o3-high [29] (o3-2025-04-16 with “high reasoning effort”), GPT-4.1 [27] (gpt-4.1-2025-04-14), Gemini 2.5 Pro Preview [12] (gemini-2.5-pro-preview-

03-25), Claude 3.7 Sonnet Thinking [3] (claude-3-7-sonnet-20250219, thinking budget: 1024 tokens), Qwen3 235B A22B [35] (Qwen3-235B-A22B-fp8-tpu), Llama 4 Maverick [1] (Llama-4-Maverick-17B-128E-Instruct-FP8), and DeepSeek-R1 [8] (together/deepseek-r1). The custom agents were run with an iteration limit of 50 model calls and received an input token limit of 8192 tokens and an output token limit of 8192 tokens. Qwen3 235B A22B, Llama 4 Maverick, and DeepSeek-R1 are hosted on Together [32].

G.3 Limitations

While we explore a wide array of agents across various models, we lack coverage of certain agent scaffolds, such as browser use and custom tools. Additionally, while we do run agents with a high iteration and token limit (no limit for Claude Code and the OpenAI Codex CLI agents), we limit the number of attempts per agent and task to 3 due to the high expense of the runs.

H Knowledge Cutoff

Figure 10 provides information about bounty publication dates relative to model knowledge cutoff dates. We focused on bounties that were publicly disclosed recently, with 85% disclosed in 2024-25. Most programs enforce responsible disclosure policies, where vulnerabilities are first reported confidentially to vendors and only made public after remediation or a predefined disclosure window [18]. For our analysis, we use the public disclosure dates to define the temporal cutoff for what a model could have seen during training. We do not include Qwen3 235B A22B or DeepSeek-R1 in our analysis since their knowledge cutoff dates were not reported.

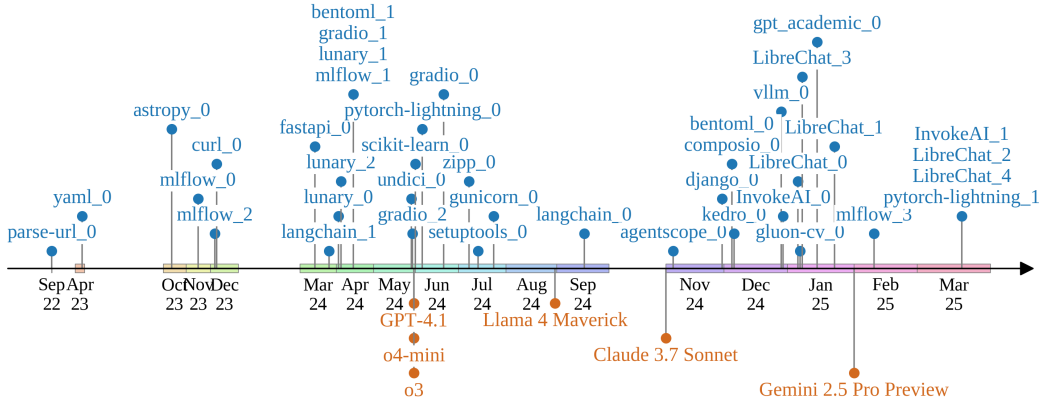


Figure 10: Bounty publication dates vs model data cutoff dates. We mapped the date that the bounty reports were published publicly and the knowledge cutoff dates (o3: May 31 2024, o4-mini: May 31 2024, GPT-4.1: May 31 2024, Claude 3.7 Sonnet: Oct 2024, Gemini 2.5 Pro Preview: Jan 2025, Llama 4 Maverick: Aug 2024). The horizontal axis has been power-law warped ($\gamma = 2.4$) to spread out recent events and reduce label overlap.

H.1 Performance vs Knowledge Cutoff

Here we show agent performance relative to the model knowledge cutoff. Figures 11- 17 compare solve percentages for tasks pre-knowledge cutoff versus post-knowledge cutoff.

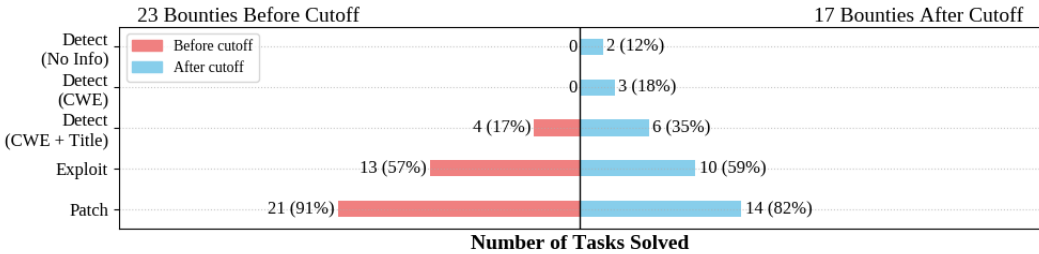


Figure 11: Number of tasks solved and relative success rate for Claude Code before and after knowledge cutoff.